

Strings

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

- | | |
|---------------------------------------|--------------------------------|
| 1. Characters in C | 5. String Library Functions |
| 2. Introduction to Strings | 6. String Problems |
| 3. String Input/Output | 7. Formatted String Operations |
| 4. String Length and Basic Operations | 8. Summary |

Characters in C



What is a Character?

Definition

A **character** is a single symbol that can be a letter, digit, punctuation mark, or special symbol. In C, characters are stored as small integers using their ASCII codes.

The char Data Type:

- Occupies **1 byte** (8 bits) of memory
- Can store values from -128 to 127 (signed) or 0 to 255 (unsigned)
- Each character has a corresponding **ASCII value**

Character Literals:

- Written using **single quotes**: 'A', 'z', '5', '\$'
- Special characters use **escape sequences**: '\n', '\t', '\0'

ASCII Table (Common Characters)

Char	ASCII	Char	ASCII	Char	ASCII
'0'	48	'A'	65	'a'	97
'1'	49	'B'	66	'b'	98
'2'	50	'C'	67	'c'	99
...	...	...	...	...	...
'9'	57	'Z'	90	'z'	122
' ' (space)	32	'\n'	10	'\0'	0

- Digits '0'-'9' have consecutive ASCII values (48-57)
- Uppercase letters 'A'-'Z' are consecutive (65-90)
- Lowercase letters 'a'-'z' are consecutive (97-122)
- Difference between lowercase and uppercase: 32 ('a' - 'A' = 32)

Working with Characters

```
#include <stdio.h>

int main() {
    char ch = 'A';

    // Print as character and as integer
    printf("Character: %c\n", ch); // Output: A
    printf("ASCII value: %d\n", ch); // Output: 65

    // Characters can be used in arithmetic
    char next = ch + 1; // 'B' (65 + 1 = 66)
    printf("Next character: %c\n", next); // Output : B

    // Convert lowercase to uppercase
    char lower = 'h';
    char upper = lower - 32; // or lower - ('a' - 'A ')
    printf("%c -> %c\n", lower, upper); // Output: h -> H

    return 0;
}
```

Character Input/Output

Reading a Single Character:

```
char ch;  
scanf ("%c", &ch); // Reads one character (including spaces)  
// Alternative : getchar () function  
char ch = getchar(); // Reads one character from input
```

Printing a Single Character:

```
char ch = 'X';  
printf ("%c", ch); // Prints the character  
// Alternative : putchar () function  
putchar (ch); // Prints one character
```

Watch Out!

`scanf("%c", &ch)` reads the next character in the input buffer, including newline characters left from previous inputs. Use `scanf(" %c", &ch)` (with a space before %c) to skip whitespace.

Character Classification Functions (<ctype.h>)

The <ctype.h> header provides useful functions for character classification:

Classification:

- isalpha(c) – Is letter?
- isdigit(c) – Is digit?
- isalnum(c) – Is letter or digit?
- isspace(c) – Is whitespace?
- isupper(c) – Is uppercase?
- islower(c) – Is lowercase?

Conversion:

- toupper(c) – Convert to uppercase
- tolower(c) – Convert to lowercase

```
#include <ctype.h>
char ch = 'a';
if (isalpha(ch)) {
    ch = toupper(ch); // ch is now 'A'
}
```


Introduction to Strings



What is a String?

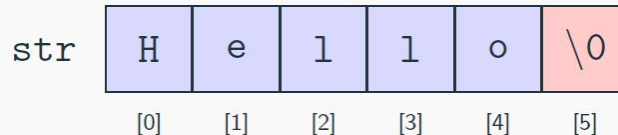
Definition

A string in C is an array of characters terminated by a null character ('`\0`').

Key Points:

- C does not have a dedicated string data type
- Strings are stored as character arrays
- The null character '`\0`' (ASCII value 0) marks the end
- Array size must be at least length + 1 (for the null terminator)

The string "Hello" in memory:



Array size: 6 (5 characters + 1 null terminator)

String Literals vs Character Literals

Understanding the Difference:

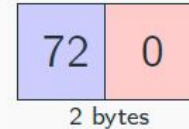
Character Literal: 'H'

- Single quotes
- One byte in memory
- Stores ASCII value (72)



String Literal: "H"

- Double quotes
- Two bytes in memory
- Stores ASCII for each character: 'H', '\0'



Important

'H' and "H" are not the same!

- 'H' is a char with value 72
- "H" is a char[2] array containing {'H', '\0'}

Declaring and Initializing Strings

Method 1: Specify Size and Initialize

```
char str1[6] = "Hello"; // Size 6: 5 chars + null terminator
```

Method 2: Let Compiler Determine Size

```
char str2[] = "Hello"; // Size automatically set to 6
```

Method 3: Specify Larger Size

```
char str3[20] = "Hello"; // Extra space filled with '\0'
```

Method 4: Character-by-Character

```
char str4[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Method 5: Pointer to String Literal (Read-only!)

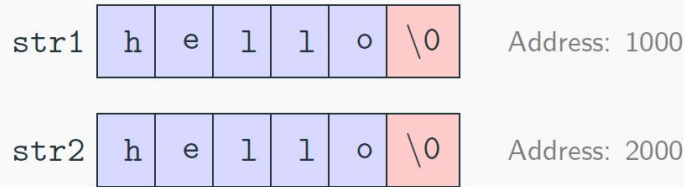
```
char *str5 = "Hello"; // Points to string in read-only memory
```

Warning: Array Bounds

Don't forget the null terminator! Without it, string functions won't know where the string ends.

String Comparison: A Common Mistake

Can we compare strings using `==`?



`(str1 == str2)` compares addresses, not contents!

Result

Even though both strings contain "hello", the comparison returns `FALSE` because they are stored at different memory locations (1000 vs 2000).

Solution: Use `strcmp()` function (covered later)

Modifying String Contents

Strings are just arrays – we can modify individual characters:

```
#include <stdio.h>
int main() {
    char str[6] = "hello";
    printf ("Original : %s\n", str); // hello
    // Change first character
    str[0] = 'y';
    printf ("After str [0] = 'y': %s\n", str); // yello
    // Truncate the string by placing null terminator
    str[4] = '\\0 ';
    printf ("After str [4] = '\\\\0': %s\n", str); // yell
    return 0;
}
```

Important

Replacing the null terminator with a non-null character creates an unterminated string, causing undefined behavior when using string functions!

String Input/Output



String Output with printf()

Using %s format specifier:

```
char name[] = "Alice";

printf("%s\n", name); // Output: Alice

// With width specification (right-aligned)
printf("%10s\n", name); // Output: Alice

// Left-aligned with width
printf ("%%-10s\n", name); // Output: Alice

// Limit characters printed
printf ("%%.3s\n", name); // Output: Ali
```

How it works: printf prints characters starting from the given address until it encounters '\0'.

String Input with scanf()

Basic Usage:

```
char name [20];  
scanf ("%s", name ); // Note: no & needed (array name IS an address)
```

Behavior:

- Skips leading whitespace
- Reads until next whitespace (space, tab, newline)
- Automatically adds null terminator

Problem: No bounds checking! If input is too long, buffer overflow occurs.

Solution: Specify maximum width:

```
char name[11];  
scanf("%10s", name); // Read at most 10 chars (leave 1 for '\0 ')
```

Limitation

`scanf ("%s")` cannot read strings with spaces. Input “Hello World” would only read “Hello”.

Reading Strings with Spaces: gets() and fgets()

The gets() function (Deprecated/Dangerous):

```
char line[80];  
gets(line); // Reads until newline, no bounds checking!
```

The fgets() function (Safe Alternative):

```
char line[80];  
fgets(line, 80, stdin); // Reads at most 79 chars + '\0'
```

Parameters:

1. `line` – destination array
2. `80` – maximum characters to read (including `'\0'`)
3. `stdin` – input stream (standard input)

Limitation

`fgets()` keeps the newline character `'\n'` in the string if there's room. You may need to remove it manually!

Removing Newline from fgets() Input

Problem: After fgets(), the string might contain ‘\n’

```
#include <stdio.h>
#include <string.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    fgets(name, 50, stdin);

    // Remove trailing newline if present
    int len = strlen(name);
    if(len > 0 && name[len - 1] == '\n') {
        name [len - 1] = '\0 ';
    }

    printf("Hello, %s!\n", name);
    return 0;
}
```

String Output: puts() and fputs()

The puts() function:

```
char message[] = "Hello, World!";  
puts(message); // Prints string and adds newline automatically
```

Comparison with printf():

- puts() automatically adds '\n' at the end
- puts() is simpler and slightly faster for plain strings
- printf() offers more formatting control

The fputs() function:

```
fputs(message, stdout); // Like puts() but NO automatic newline
```

Advanced: Reading Until a Delimiter with scanf()

Using %[^\n] format:

```
char line[80];

// Read until newline ( similar to fgets but with scanf )
scanf("%[^\n]", line); // Read everything except newline

// Read until comma
scanf("%[^,]", line); // Stops at first comma

// With width limit
scanf("%79[^\n]", line ); // At most 79 chars, stop at newline
```

Flushing remaining input:

```
// After reading a line , clear the rest including newline
scanf("%79[^\n]%*[^\\n]", s);
scanf("%*c"); // Consume the newline
```

String Length and Basic Operations



Finding String Length: strlen()

strlen() Function

Returns the number of characters in a string, **not including** the null terminator.

```
#include <stdio.h>
#include <string.h>

int main () {
    char str1[] = "Hello";
    char str2[] = "Hello World";
    printf("Length of \"%s\": %lu\n", str1, strlen(str1)); // 5
    printf("Length of \"%s\": %lu\n", str2, strlen(str2)); // 11
    // Compare with sizeof
    printf ("sizeof(str1): %lu\n", sizeof(str1)); // 6 ( includes '\0 ' )
    return 0;
}
```

Note: strlen() counts characters; sizeof() gives array size in bytes.

Implementing strlen() Ourselves

Understanding how string length is computed:

```
int myStrlen(char str[]) {  
    int length = 0;  
    // Keep counting until we hit the null terminator  
    while(str[length] != '\0') {  
        length++;  
    }  
    return length;  
}  
  
int main() {  
    char word[] = "Programming";  
    printf("Length: %d\n", myStrlen(word)); // Output : 11  
    return 0;  
}
```

Key Insight: The null terminator ‘\0’ is the sentinel that marks where the string ends.

Problem: Print Each Character Vertically

Task: Print each character of a string on a separate line.

```
#include <stdio.h>
#include <string.h>

int main() {
    char sentence[80];
    int i, length;

    printf("Enter a sentence: ");
    fgets(sentence, 80, stdin);

    length = strlen(sentence);

    for (i = 0; i < length; i++) {
        printf("%c\n", sentence[i]);
    }

    return 0;
}
```

Problem: Count Vowels and Consonants

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char s[80];
    int i, length, vowels = 0, consonants = 0;
    printf("Enter a sentence: ");
    fgets(s, 80, stdin);
    length = strlen(s);
    for (i = 0; i < length; i++) {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            vowels++;
        else if (isalpha (c))
            consonants++;
    }
    printf("Vowels: %d, Consonants: %d\n", vowels, consonants);
    return 0;
}
```

String Library Functions



Overview of <string.h> Functions

Algorithm	Purpose
<code>strlen(s)</code>	Get length of string
<code>strcpy(dest, src)</code>	Copy string
<code>strncpy(dest, src, n)</code>	Copy at most n characters
<code>strcat(dest, src)</code>	Concatenate (append) strings
<code>strncat(dest, src, n)</code>	Concatenate at most n characters
<code>strcmp(s1, s2)</code>	Compare two strings
<code>strncmp(s1, s2, n)</code>	Compare first n characters
<code>strchr(s, c)</code>	Find first occurrence of character
<code>strrchr(s, c)</code>	Find last occurrence of character
<code>strstr(s1, s2)</code>	Find substring

Remember: Always `#include <string.h>` to use these functions!

String Copy: strcpy() and strncpy()

Why can't we just use =?

- `str1 = str2;` copies the **pointer**, not the content. We need to copy character by character

Using strcpy():

```
char src[] = "Hello";  
char dest[20];  
strcpy(dest, src); // Copy src into dest  
printf("%s\n", dest); // Output: Hello
```

Using strncpy() (safer):

```
char src [] = " Hello World ";  
2 char dest [6];  
4 strncpy (dest , src , 5); // Copy at most 5 characters  
5 dest [5] = '\\0 ' ; // Manually add null terminator !  
6 printf ("%s\n", dest ); // Output : Hello
```

Warning

`strncpy()` may NOT add a null terminator if source is longer than `n`. Always add it manually!

String Concatenation: strcat() and strncat()

Appending one string to another:

```
char greeting[50] = "Hello, ";  
char name[] = "World";  
  
strcat(greeting, name); // Append name to greeting  
printf("%s\n", greeting); // Output : Hello , World  
  
// Using strncat for safety  
char buffer[15] = "Hi, ";  
strncat(buffer, "Everyone in the class", 5); // Append at most 5 chars  
printf("%s\n", buffer); // Output : Hi , Every
```

Before:

H	e	l	l	o	\0
---	---	---	---	---	----

After:

H	e	l	l	o	W	o	r	l	d	\0
---	---	---	---	---	---	---	---	---	---	----

String Comparison: strcmp() and strncmp()

strcmp(s1, s2) Return Values

- Returns **0** if strings are equal
- Returns **< 0** if s1 comes before s2 (lexicographically)
- Returns **> 0** if s1 comes after s2

```
char s1[] = "apple";
char s2[] = "banana";
char s3[] = "apple";
printf("%d\n", strcmp(s1, s2)); // Negative (apple < banana)
printf("%d\n", strcmp(s2, s1)); // Positive (banana > apple)
printf("%d\n", strcmp(s1, s3)); // 0 (equal)
// Compare first n characters
printf("%d\n", strncmp("hello", "help", 3)); // 0 (hel == hel)
printf("%d\n", strncmp("hello", "help", 4)); // Negative (hell < help)
```

Comparison is based on ASCII values, character by character.

Implementing String Comparison

How does strcmp() work internally?

```
int myStrcmp(char s1[], char s2[]) {
    int i = 0;

    // Compare characters until mismatch or end
    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i]) {
            return s1[i] - s2[i]; // Return difference
        }
        i++;
    }

    // Handle different lengths
    return s1[i] - s2[i];
}

// Returns 0 if equal, negative if s1 < s2, positive if s1 > s2
```


Finding Characters: strchr() and strrchr()

Search for a character in a string:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello World";
    char *pos;
    // Find first occurrence of 'o'
    pos = strchr(str, 'o');
    if (pos != NULL) {
        printf("First 'o' at position: %ld\n", pos - str); // 4
    }
    // Find last occurrence of 'o'
    pos = strrchr(str, 'o');
    if (pos != NULL) {
        printf("Last 'o' at position: %ld\n", pos - str); // 7
    }
    return 0;
}
```

Finding Substrings: strstr()

Search for a substring within a string:

```
#include <stdio.h>
#include <string.h>
int main () {
    char sentence[] = "The quick brown fox jumps";
    char *pos;
    // Find "brown" in the sentence
    pos = strstr(sentence, "brown");
    if (pos != NULL) {
        printf("Found at position: %ld\n", pos - sentence); // 10
        printf("Substring from match: %s\n", pos); // brown fox jumps
    } else {
        printf("Not found\n");
    }
    return 0;
}
```

Returns: Pointer to first occurrence, or NULL if not found.

String Problems



Problem: Check if Two Strings are Equal

Task: Write a function to check if two strings are the same.

```
int sameString(char s1[], char s2[]) {  
    int i;  
  
    // If lengths differ, strings are not equal  
    if (strlen(s1) != strlen(s2)) {  
        return 0;  
    }  
    // Compare character by character  
    for (i = 0; i < strlen(s1); i++) {  
        if (s1[i] != s2[i]) {  
            return 0; // Mismatch found  
        }  
    }  
    return 1; // All characters matched  
}
```

Note: In practice, just use `strcmp(s1, s2) == 0`

Problem: Find Last Position of a Character

Task: Find the last position where a character appears in a string.

```
#include <stdio.h>
#include <string.h>
int main() {
    char sentence[80];
    char target;
    int i, length, lastPos = -1;
    printf("Enter a sentence: ");
    fgets(sentence, 80, stdin);
    printf("Enter character to find: ");
    scanf(" %c", &target);
    length = strlen(sentence);
    for (i = 0; i < length; i++) {
        if (sentence[i] == target) lastPos = i;
    }
    if (lastPos == -1) printf("Not found\n");
    else printf("Last occurrence at position: %d\n", lastPos);
    return 0;
}
```

Problem: Count Word Occurrences

Task: Count how many times a letter appears in a sentence.

```
#include <stdio.h>
#include <string.h>
int main() {
    char sentence[80];
    char target;
    int i, count = 0;
    printf("Enter a sentence: ");
    fgets(sentence, 80, stdin);
    printf("Which letter? ");
    scanf(" %c", &target);
    for (i = 0; i < strlen(sentence); i++) {
        if (sentence[i] == target) count++;
    }
    if (count == 0) printf("Not found\n");
    else printf("Found %d times\n", count);
    return 0;
}
```

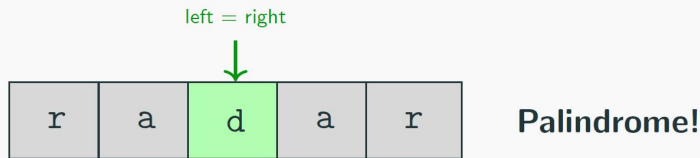
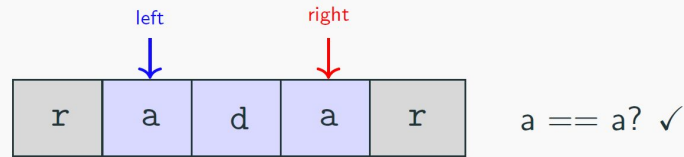
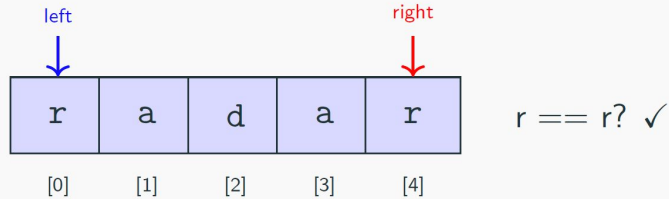
Problem: Check for Palindrome

Task: Check if a word reads the same forwards and backwards

```
#include <stdio.h>
#include <string.h>
int isPalindrome(char s[]) {
    int left = 0;
    int right = strlen(s) - 1;
    while (left < right) {
        if (s[left] != s[right]) return 0; // Not a palindrome
        left++;
        right--;
    }
    return 1; // Is a palindrome
}
int main() {
    char word[] = "radar";
    if (isPalindrome(word)) printf("%s is a palindrome\n", word);
    else printf ("%s is not a palindrome\n", word);
    return 0;
}
```

Palindrome Check: Visualization

Checking if “radar” is a palindrome:



Problem: Reverse a String

Task: Reverse a string in-place (without using extra array).

```
#include <stdio.h>
#include <string.h>
void reverseString(char s[]) {
    int left = 0, right = strlen(s) - 1;
    while(left < right) {
        // Swap characters
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
}
int main () {
    char str[] = "Hello";
    reverseString(str);
    printf("%s\n", str); // Output : olleH
    return 0;
}
```

Problem: Find a Word in a Sentence

Task: Find the first position of a word within a sentence.

```
#include <stdio .h>
#include <string .h>
int findWord(char sentence[], char word[]) {
    int i, j;
    int sentLen = strlen(sentence);
    int wordLen = strlen(word);
    for (i = 0; i <= sentLen - wordLen; i++) {
        for (j = 0; j < wordLen; j++) {
            if (sentence[i + j] != word[j])
                break;
        }
        if (j == wordLen)
            return i; // Found at position i
    }
    return -1; // Not found
}
```

Note: The `strstr()` function does this more efficiently!

Formatted String Operations



Writing to Strings: sprintf()

strcmp(s1, s2) Return Values

Works like scanf(), but reads from a string instead of standard input.

```
#include <stdio.h>
int main () {
    char data[] = "John 25 3.75";
    char name[20];
    int age;
    float gpa;
    // Parse data from string
    sscanf(data, "%s %d %f", name, &age, &gpa);
    printf("Name: %s\n", name); // Name : John
    printf("Age: %d\n", age); // Age : 25
    printf("GPA: %.2f\n", gpa); // GPA : 3.75
    return 0;
}
```

Useful for: Parsing lines read from files, processing command-line arguments.

Summary



Key Takeaways

1. **Characters** are stored as ASCII values in char (1 byte)
2. **Strings** are character arrays terminated by ‘\0’
3. String literals use **double quotes** ("Hello")
4. Character literals use **single quotes** ('H')
5. strcmp() compares strings; == compares addresses
6. Use fgets() instead of gets() for safe input
7. strlen() returns length; sizeof() includes null terminator
8. strcpy() copies; strcat() concatenates
9. strchr() finds characters; strstr() finds substrings
10. sprintf()/sscanf() work with strings instead of I/O

Common Mistakes to Avoid

String Pitfalls

1. **Forgetting null terminator** – causes undefined behavior
2. **Buffer overflow** – writing beyond array bounds
3. **Using == to compare strings** – compares addresses!
4. **Modifying string literals** – they may be read-only
5. **Forgetting to allocate space** for null terminator
6. **Using** `gets()` – dangerous, use `fgets()`
7. `strncpy()` **not adding null** – may not terminate

Golden Rule: Always ensure strings are properly null-terminated and stay within array bounds

Quick Reference: String Functions

Operation	Function	Example
Length	<code>strlen(s)</code>	Returns 5 for "Hello"
Copy	<code>strcpy(dest, src)</code>	Copies s into d
Safe Copy	<code>strncpy(dest, src, n)</code>	Copies at most n chars
Concatenate	<code>strcat(dest, src)</code>	Appends s to d
Compare	<code>strcmp(s1, s2)</code>	0 if equal
Find Char	<code>strchr(s, c)</code>	Pointer to first c
Find Substring	<code>strstr(s1, s2)</code>	Pointer to sub in s

Header: `#include <string.h>`

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET